

浙江大学

本科实验报告

课程名称： 电子电路设计实验 II

实验名称： 多功能数字时钟的设计与制作

姓 名：

学 院： 信息与工程学院

专 业： 信息工程

学 号：

指导教师： 施红军、叶险峰、邓靖靖

目录

1	实验目的	1
2	实验任务与要求	1
3	系统总体方案	2
3.1	总体结构	2
3.2	工作模式设计	2
4	硬件模块设计	3
4.1	主要器件选择	3
4.2	关键模块说明	3
4.3	引脚分配	3
4.4	温度采集参数计算	7
5	软件设计	7
5.1	工程结构	7
5.2	主程序结构	7
5.3	软件流程	8
5.4	按键与设置逻辑	8
5.5	时间保存与 EEPROM 设置保存	9
5.6	闹钟与报警设计	9
5.7	扩展功能设计	9
6	程序关键代码说明	9
6.1	外设对象与引脚定义	9
6.2	模式枚举与设置结构	9
7	调试过程与结果	10
7.1	编译与下载	10
7.2	功能测试表	11
8	问题分析与改进	15
9	实验总结	15
10	参考资料	15
A	附录：项目主要代码	16
1.1	main.cpp	16
1.2	ClockPlus.h	16
1.3	ClockPlus.cpp	16

浙江大学 实验报告

专业： 信息工程
姓名： _____
学号： _____
日期： 2026.5.26
地点： 东四 216

课程名称： 电子电路设计实验 II 指导老师： 施红军、叶险峰、邓靖靖 成绩： _____
实验名称： 多功能数字时钟的设计与制作 实验类型： 设计性实验 同组学生姓名： _____

一、实验目的

本实验以 Arduino UNO 为控制核心，设计并调试一个多功能数字时钟系统。系统通过 DS1302 实时时钟模块完成计时和掉电走时，通过 LCD1602 完成人机显示，通过 LM35 温度传感器完成温度检测，并利用按键、蜂鸣器实现时间设置、闹钟、整点响铃、温度报警等功能。在完成基本要求的基础上，程序进一步加入秒表、倒计时、番茄钟、温度历史、动画显示与多组闹钟等扩展功能，使系统具有更完整的人机交互体验。

1. 学习掌握 Arduino UNO 与 DS1302、LCD1602、LM35 等常用外设的连接和软件驱动方法。
2. 掌握多功能数字时钟的硬件设计、器件选型、接口分配和参数计算方法。
3. 掌握基于状态机的软件设计思想，完成时间显示、日期显示、参数设置、闹钟和报警等功能。
4. 学习 PlatformIO 工程管理、模块化程序编写、编译下放和串口调试方法。
5. 通过扩展功能设计，提高系统的可玩性、可演示性与综合设计完整度。

二、实验任务与要求

根据实验任务书，本实验需要完成如下功能：

1. 显示时间、日期和星期。
2. 断电保存时间，即主电源断开后，DS1302 依靠备用电池继续维持内部时钟运行，上电后 Arduino 重新读取当前时间。
3. 通过按键设置时间和日期。
4. 实现整点响铃。
5. 实现自定义闹钟。
6. 显示温度。
7. 实现自定义报警温度。
8. 按选择键进入时间设置功能；同时按加、减键进入闹钟和报警温度设置功能。
9. 在设置状态下，再按选择键移动光标；光标跳到哪，当前参数即可通过加、减键修改。

本设计在保留上述功能的前提下，增加了以下扩展功能：三组闹钟、工作日/周末闹钟模式、贪睡功能、秒表、倒计时、番茄钟、温度最高/最低值统计、温度历史条形图、LCD 自定义字符动画、按键音效、开机动画和隐藏彩蛋。

三、系统总体方案

3.1 总体结构

系统由 Arduino UNO、DS1302 实时时钟模块、LCD1602 显示模块、LM35 温度传感器、三枚按键和有源/无源蜂鸣器组成。Arduino UNO 负责读取 RTC 时间和温度数据，扫描按键状态，根据当前工作模式刷新 LCD 显示，并在闹钟、整点或温度超限时驱动蜂鸣器。

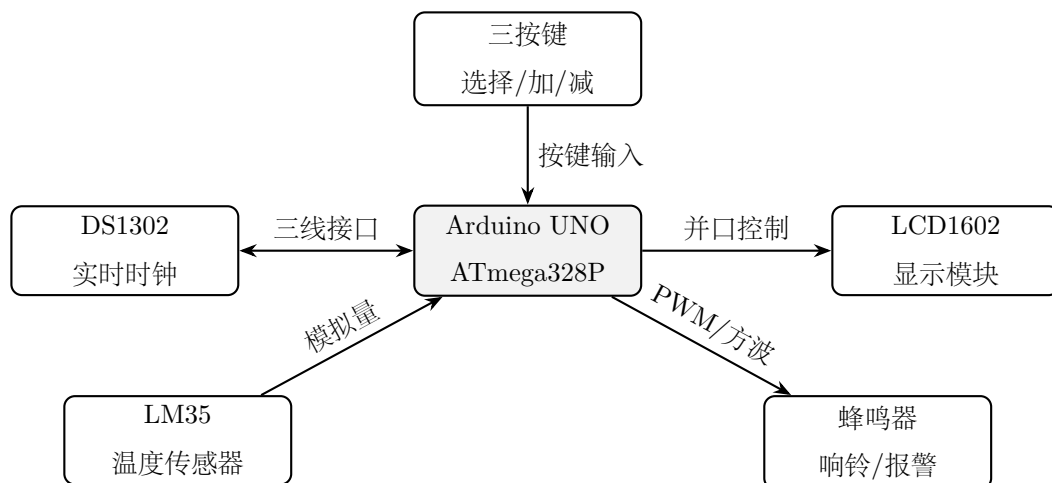


图 1: 多功能数字时钟系统硬件框图

3.2 工作模式设计

程序采用多模式菜单结构。正常运行时，选择键用于在时钟、温度、秒表、倒计时、番茄钟、温度历史等界面之间切换；长按或组合按键进入设置状态；加、减键用于修改当前光标对应的参数。

表 1: 系统工作模式

序号	模式	主要功能
1	时钟模式	显示时分秒、日期、星期，检测整点响铃和闹钟
2	温度模式	显示当前温度、最高温、最低温和温度报警状态
3	秒表模式	通过按键启动、暂停、清零，适合展示扩展交互
4	倒计时模式	设置倒计时时间，计时结束后蜂鸣提醒
5	番茄钟模式	按工作时间/休息时间交替提醒
6	历史模式	显示温度历史条形图和趋势

四、硬件模块设计

4.1 主要器件选择

表 2: 主要器件及作用

模块	型号/器件	作用
主控	Arduino UNO	负责外设控制、按键扫描、显示刷新与蜂鸣器驱动
实时时钟	DS1302	保存秒、分、时、日、月、星期、年等时间信息，并在备用电池供电下维持走时
显示	LCD1602	显示时间、日期、星期、温度、设置菜单和扩展功能界面
温度检测	LM35	输出与摄氏温度近似线性对应的模拟电压，用于室温检测
输入	三枚按键	选择、加、减，实现菜单切换和参数设置
输出	蜂鸣器	用于整点响铃、闹钟、倒计时、按键音和报警提示

4.2 关键模块说明

DS1302 是低功耗实时时钟芯片，具有主电源和备用电源引脚，可在主电源断开时依靠备用电池继续维持时间信息；其还包含静态 RAM，并支持字节传输和突发传输。LCD1602 模块一般采用兼容 HD44780 的控制芯片，Arduino 的 `LiquidCrystal` 库可通过 4 位或 8 位方式驱动该类字符液晶。LM35 传感器的输出电压与摄氏温度成正比，典型比例系数为 $10\text{ mV}/^{\circ}\text{C}$ 。

4.3 引脚分配

本系统为了避免 Arduino UNO 引脚冲突，对 LCD、DS1302、按键、温度传感器和蜂鸣器进行统一规划，如表所示。

表 3: Arduino UNO 引脚分配表

Arduino 引脚	连接模块	说明
D12	LCD RS	LCD 寄存器选择端
D11	LCD E	LCD 使能端
D5、D4、D3、D2	LCD D4-D7	LCD 4 位数据总线
D8	DS1302 CE/RST	DS1302 片选/复位端
D9	DS1302 IO	DS1302 数据端
D10	DS1302 SCLK	DS1302 时钟端
D13	Buzzer	蜂鸣器控制端
A0	CHOOSE	选择键
A1	ADD	加键
A2	MINUS	减键
A3	LM35 OUT	温度模拟输入

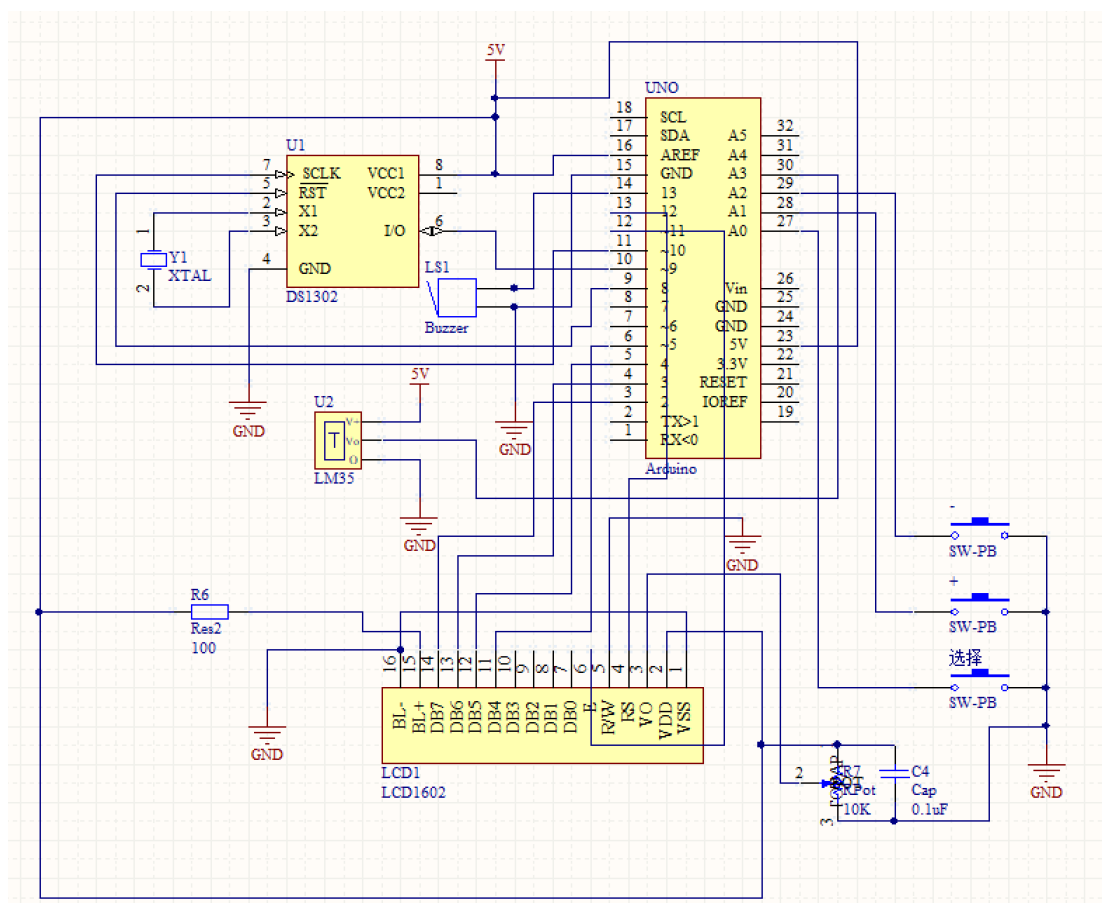


图 2: 多功能时钟电路原理图

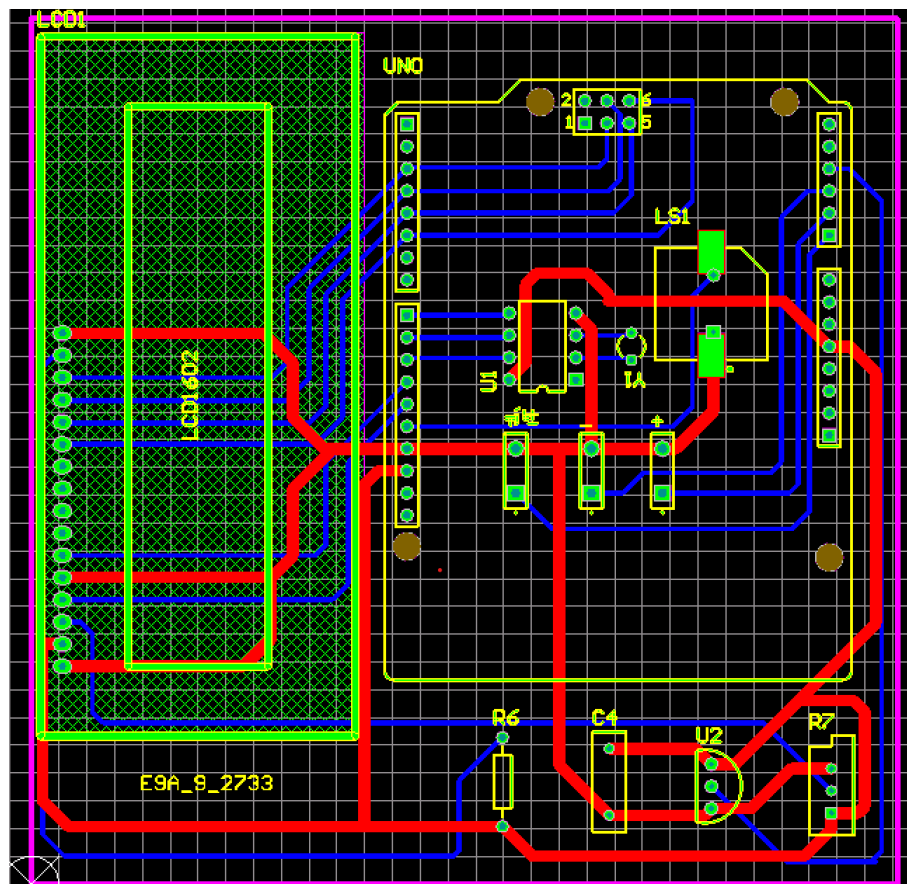


图 3: 多功能时钟电路 PCB 图

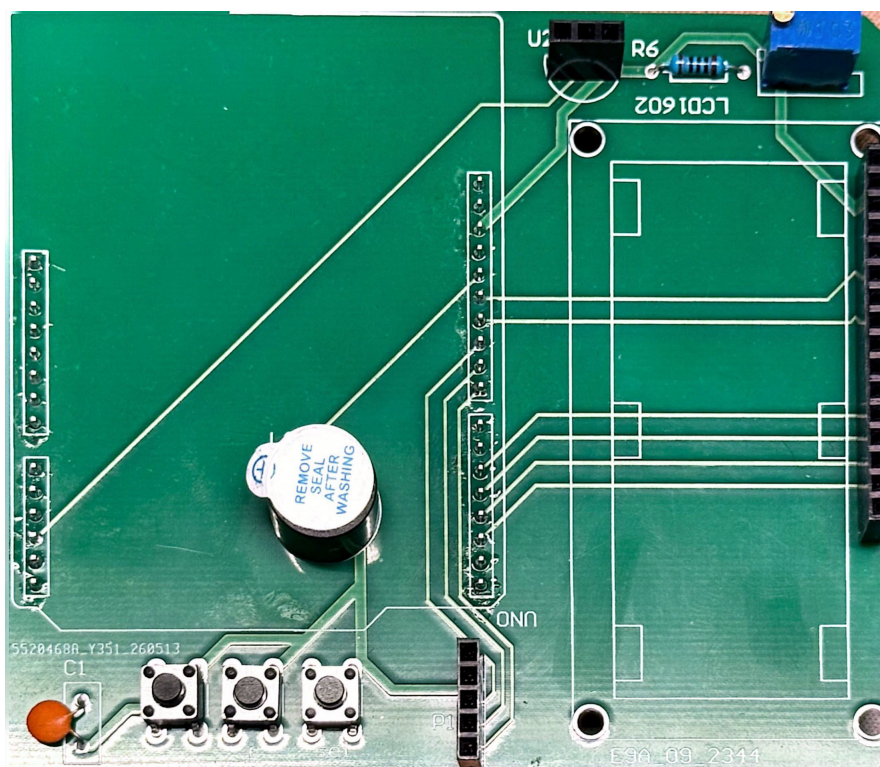


图 4: 多功能时钟电路 PCB 实物图

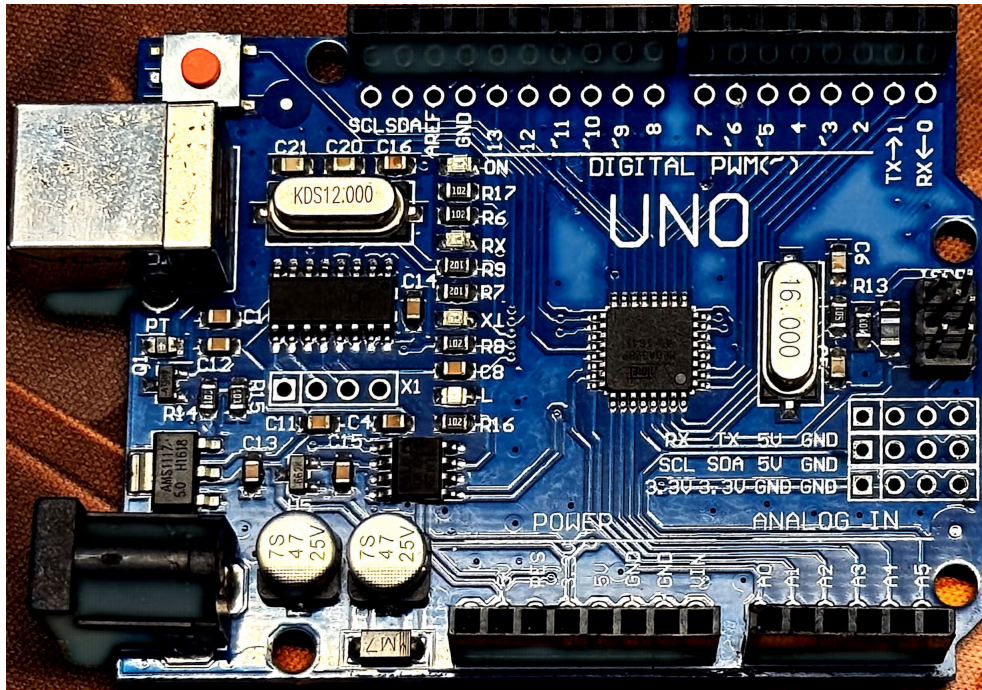


图 5: Auduino UNO 实物图

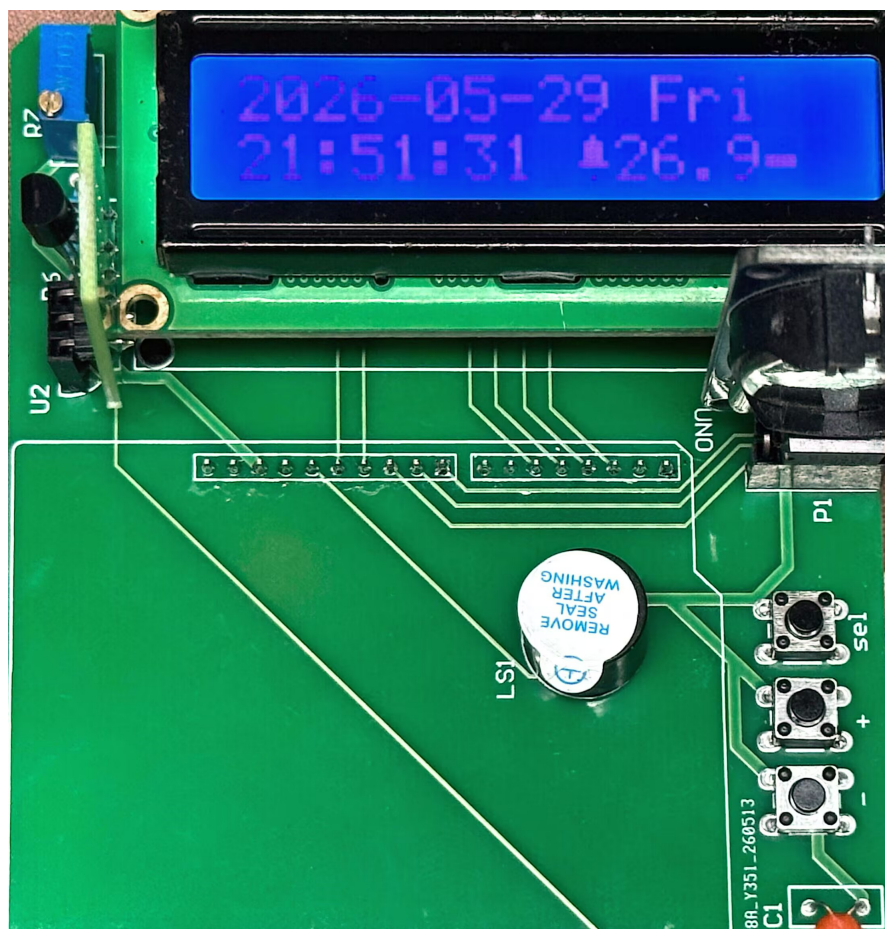


图 6: 连接完成的版图

4.4 温度采集参数计算

Arduino UNO 默认模拟参考电压为 5 V, ADC 为 10 位, 因此一次 ADC 数值对应的电压约为

$$U_{\text{step}} = \frac{5.0}{1024} \text{ V} \approx 4.88 \text{ mV}. \quad (4.1)$$

LM35 的典型输出灵敏度为 10 mV/°C, 因此温度计算式为

$$T = \frac{\text{ADC} \times 5.0}{1024} \times 100. \quad (4.2)$$

为减小显示抖动, 程序中对多次采样结果求平均, 再更新当前温度、最高温和最低温。

五、软件设计

5.1 工程结构

本项目采用 PlatformIO 工程结构, 源代码进行模块化组织, 主函数仅调用时钟系统初始化和循环函数。

表 4: PlatformIO 工程文件结构

文件	作用
platformio.ini	配置 Arduino UNO 开发板、Arduino 框架与依赖库
src/main.cpp	Arduino 入口文件, 调用 <code>clockPlusSetup()</code> 和 <code>clockPlusLoop()</code>
src/ClockPlus.h	模块头文件, 声明对外接口并包含外设库
src/ClockPlus.cpp	系统主体程序, 包含按键、显示、RTC、温度、闹钟和扩展功能逻辑

5.2 主程序结构

Listing 1: 主程序 main.cpp

```
1 #include <Arduino.h>
2 #include "ClockPlus.h"
3
4 void setup() {
5     clockPlusSetup();
6 }
7
8 void loop() {
9     clockPlusLoop();
10 }
```

其中 `setup` 为初始化, `loop` 为程序循环执行, `ClockPlus.h` 封装了时钟的核心函数。

5.3 软件流程

程序初始化阶段完成 LCD、按键、蜂鸣器、DS1302 和 EEPROM 参数初始化；循环阶段主要完成按键扫描、时间读取、温度采样、模式切换、显示刷新、闹钟判断和蜂鸣器控制。整体流程如图所示。

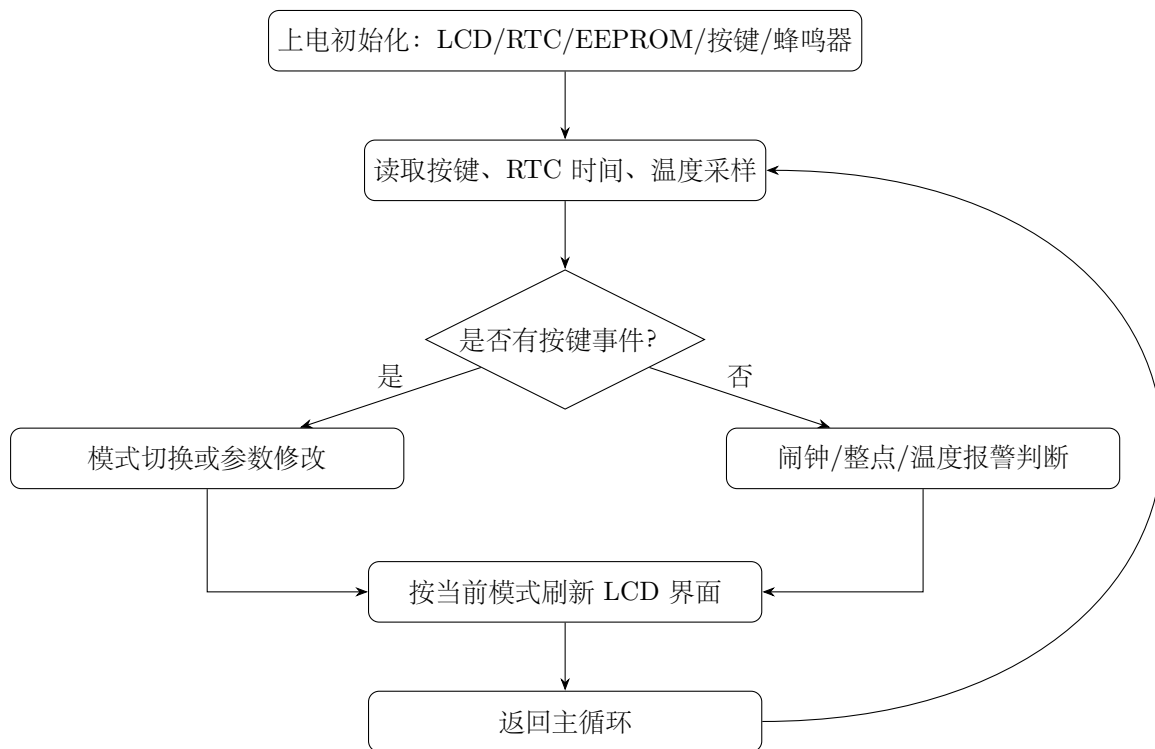


图 7: 主程序流程图

5.4 按键与设置逻辑

系统使用三个按键实现多功能菜单。为了避免机械抖动导致一次按下被识别多次，程序使用时间间隔进行软件消抖。按键操作分为短按、组合键和设置状态下的参数修改：选择键用于切换界面或移动光标，加键和减键用于参数增减，同时按加减键可进入闹钟与报警温度设置。

表 5: 按键功能规划

按键操作	功能
选择键短按	正常模式下切换界面；设置模式下移动光标
加键短按	设置模式下当前参数加一；秒表/倒计时等界面执行启动或暂停
减键短按	设置模式下当前参数减一；部分界面执行清零或复位
加键和减键	进入闹钟、报警温度等扩展设置界面
长时间无操作	自动保存设置并返回正常显示状态

5.5 时间保存与 EEPROM 设置保存

本实验中的“断电保存时间”并不是由 Arduino 在断电后继续执行程序,而是由 DS1302 依靠备用电池保持内部时钟继续运行;Arduino 再次上电后读取 DS1302 的当前时间即可。与时间不同,闹钟、温度阈值、倒计时默认值等用户设置由 EEPROM 保存,上电后读取,避免每次重新设置。

5.6 闹钟与报警设计

程序设置三组闹钟,每组包含小时、分钟、是否启用和重复类型。重复类型包括每日、工作日和周末。闹钟触发时蜂鸣器播放提示音,并支持贪睡逻辑。温度报警由用户设置阈值,当 LM35 测得温度超过阈值时,LCD 提示并驱动蜂鸣器报警。

5.7 扩展功能设计

1. **秒表**: 用于计时演示,可通过按键启动、暂停和清零。
2. **倒计时**: 用户可设置倒计时时间,倒计时结束后响铃提醒。
3. **番茄钟**: 工作时间和休息时间交替运行,适合作为学习计时器。
4. **温度历史**: 定期采样并显示温度变化条形图,增强界面表现力。
5. **LCD 动画**: 利用 LCD1602 自定义字符制作动态图标,提高演示效果。
6. **隐藏彩蛋**: 通过特殊按键序列触发提示音或特殊显示,用于展示扩展设计能力。

六、程序关键代码说明

6.1 外设对象与引脚定义

Listing 2: 引脚与外设对象定义节选

```
1 LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // RS,E,D4,D5,D6,D7
2
3 const int PIN_CHOOSE = A0;
4 const int PIN_ADD     = A1;
5 const int PIN_MINUS   = A2;
6 const int PIN_LM35    = A3;
7 const int PIN_BUZZER  = 13;
8
9 const uint8_t CE_PIN   = 8;
10 const uint8_t IO_PIN   = 9;
11 const uint8_t SCLK_PIN = 10;
12 DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN);
```

6.2 模式枚举与设置结构

Listing 3: 模式与设置数据结构节选

```
1  enum Mode : uint8_t {
2      MODE_CLOCK = 0,
3      MODE_TEMP,
4      MODE_STOPWATCH,
5      MODE_TIMER,
6      MODE_POMODORO,
7      MODE_HISTORY,
8      MODE_COUNT
9  };
10
11 struct AlarmSetting {
12     uint8_t hour;
13     uint8_t minute;
14     uint8_t enable;
15     uint8_t repeat;
16 };
17
18 struct Settings {
19     uint16_t magic;
20     AlarmSetting alarms[3];
21     int16_t tempAlarm10;
22     uint8_t timerH;
23     uint8_t timerM;
24     uint8_t timerS;
25     uint8_t pomoworkMin;
26     uint8_t pomobreakMin;
27 };
```

七、调试过程与结果

7.1 编译与下载

本项目使用 VS Code + PlatformIO 完成编译和下载，目标板配置为 Arduino UNO。核心配置如下：

Listing 4: platformio.ini 配置

```
1  [env:uno]
2  platform = atmelavr
3  board = uno
4  framework = arduino
5
6  lib_deps =
7      https://github.com/msparks/arduino-ds1302.git
8      arduino-libraries/LiquidCrystal
9
10 monitor_speed = 9600
```

7.2 功能测试表

表 6: 功能测试记录占位表

序号	测试项目	结果	说明
1	时间、日期、星期显示	通过测试	上电后 LCD 正常显示当前时间
2	断电保存时间	通过测试	断电一段时间后重新上电,时间继续运行
3	按键设置时间日期	通过测试	选择键移动光标, 加减键修改参数
4	整点响铃	通过测试	到达整点时蜂鸣器响铃
5	自定义闹钟	通过测试	到达设定时间触发闹钟
6	温度显示	通过测试	显示当前温度及最高/最低温度
7	温度报警	通过测试	超过阈值后触发提示
8	秒表/倒计时/番茄钟	通过测试	扩展功能可正常进入和操作

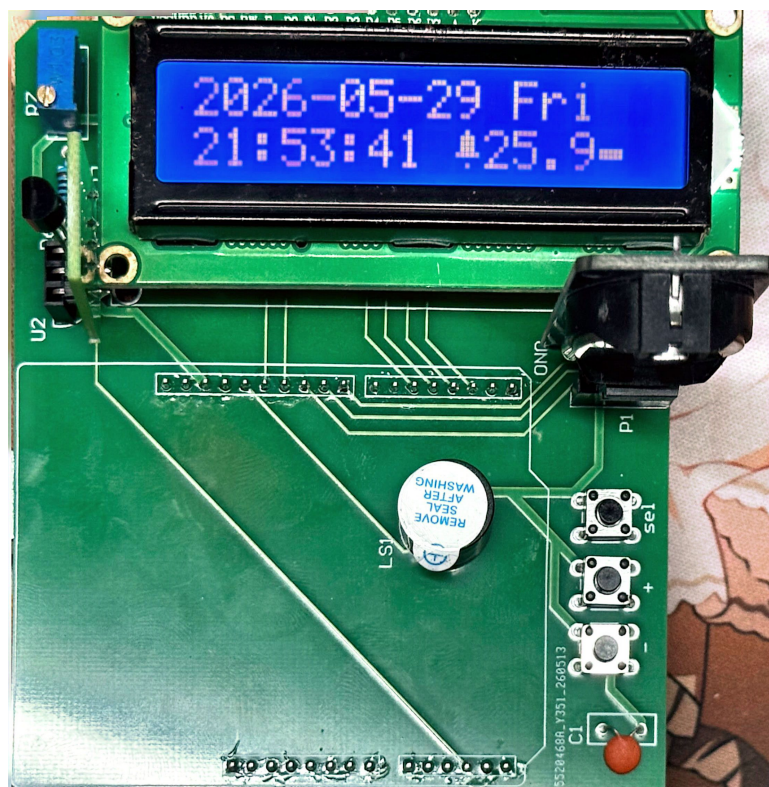


图 8: 时间显示页面, 显示格式为日期 + 时间 + 当前温度

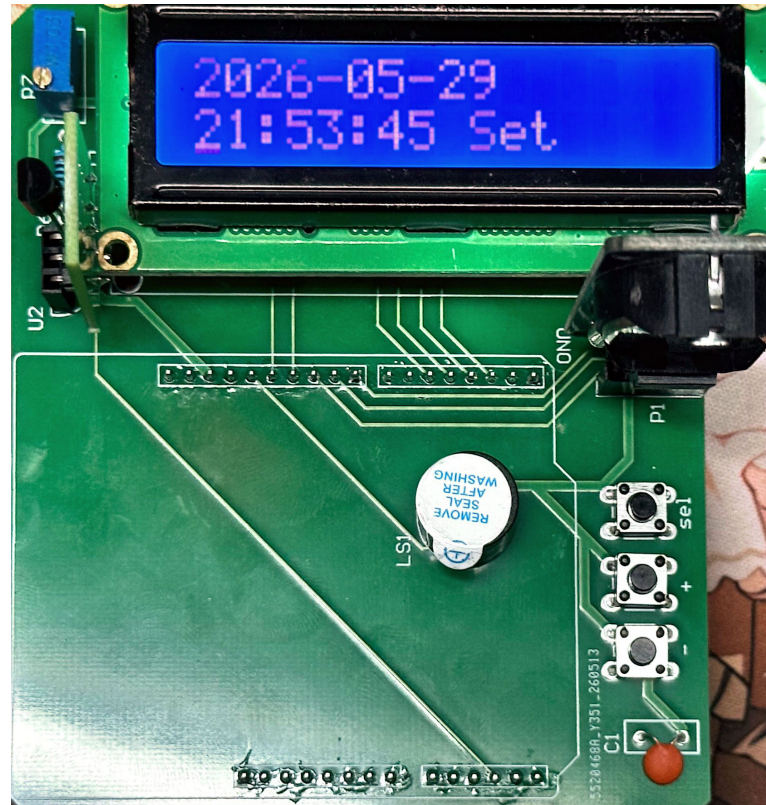


图 9: 时间设定页面, 通过 set 键可以调整修改的时间单位

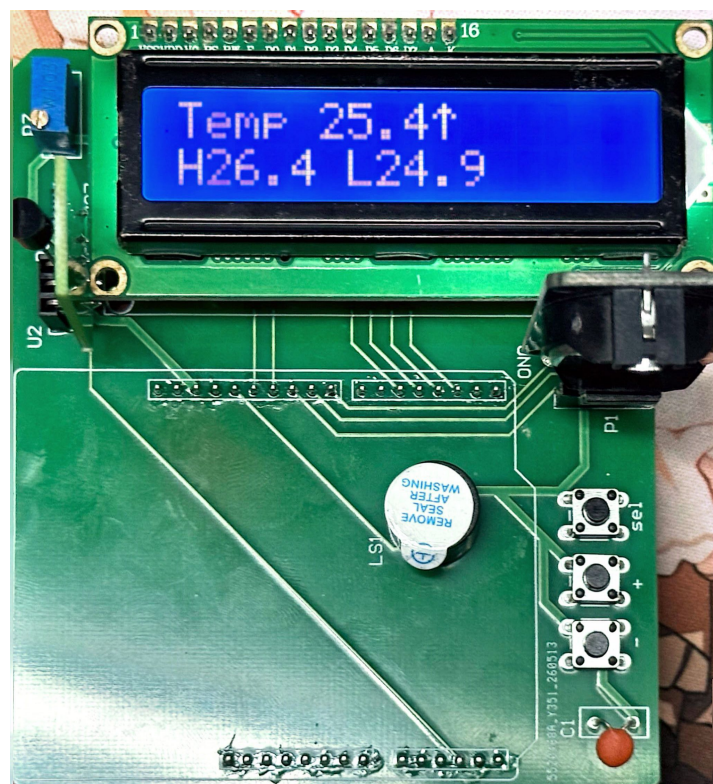


图 10: 温度显示页面, Temp 表示当前温度, ↑ 表示气温上升, H 表示记录的最高温度, L 表示记录的最低温度

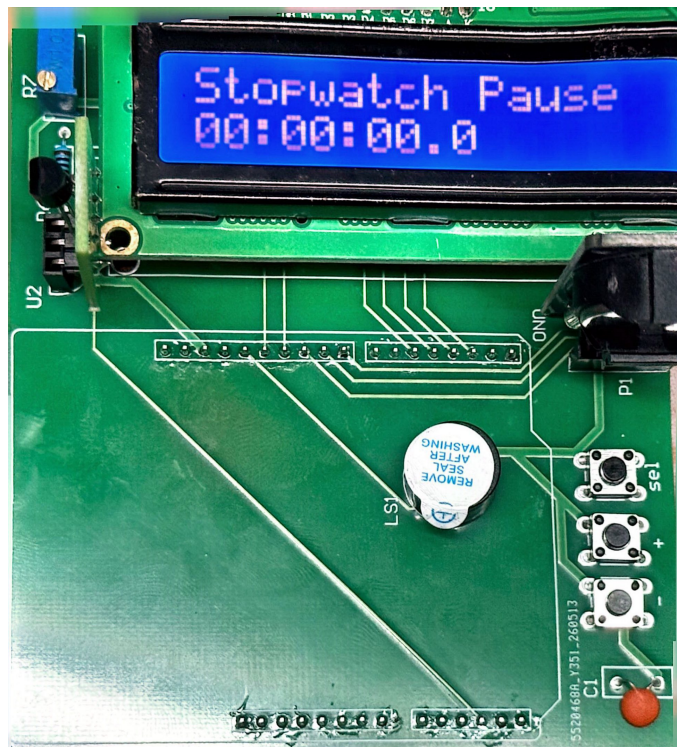


图 11: 秒表页面, 按下 set 开始计时, 再次按下 set 暂停, 按下-复位



图 12: 计时页面, 按下 set 开始倒数, 再次按下 set 暂停, 按下-复位

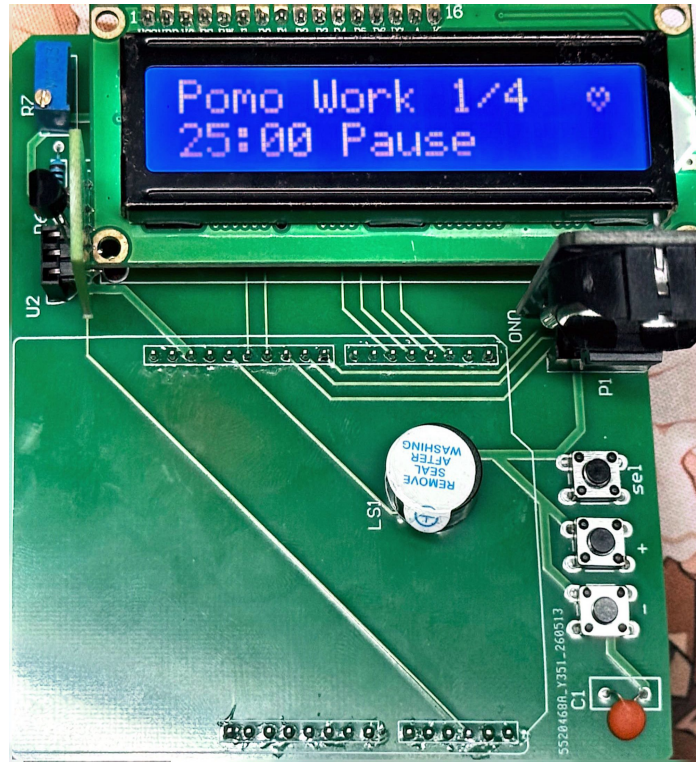


图 13: 番茄钟页面, 默认工作时长 25min



图 14: 综合设定页面, 同时按下 + 和-进入, 可以设定闹钟、报警温度阈值等

八、问题分析与改进

1. **库文件兼容问题:** 在 PlatformIO 中, 仅搜索到名称相近的 DS1302 库并不代表其适用于 Arduino UNO, 需要选择与 Arduino 框架兼容、头文件名称匹配的库。最终采用 GitHub 形式指定 DS1302 库, 并在 `platformio.ini` 中显式写入依赖。
2. **温度显示抖动:** LM35 输出模拟电压, ADC 采样易受电源和线路干扰影响。程序通过多次采样平均和最高/最低值统计改善显示效果。
3. **菜单功能较多:** 扩展功能增加后, 单个主函数过长不利于维护, 因此采用 `ClockPlus.h` 和 `ClockPlus.cpp` 进行模块化封装。

九、实验总结

本实验完成了基于 Arduino UNO 的多功能数字时钟设计。系统实现了时间、日期、星期显示, DS1302 断电走时, 按键设置时间日期, 自定义闹钟, 整点响铃, 温度显示和温度报警等基本要求。在此基础上, 进一步加入了多组闹钟、秒表、倒计时、番茄钟、温度历史、LCD 动画等扩展功能。通过本实验, 我进一步熟悉了 Arduino 外设接口、PlatformIO 工程管理、模块化程序设计和综合电子系统调试方法。

实验中我们遇到的最大问题并不像大多数人一样来自硬件, 而是来自软件。在使用 macOS Visual Studio Code 的 Platform IO 时, 由于库文件 `DS1302.h` 和 `LiquidCrystal.h` 的缺失, 程序一直没法运行, 前往 Platform IO 官网下载库文件也未解决问题。在搜索网络相关资料后, 我们发现是 Platform IO 的库文件与 Arduino 框架不兼容, 并最终改为 GitHub 库中的开源库文件。

另一大问题是软件代码的测试, 由于代码长度上千行, 一次性全部调试即使出 BUG 了也很难定位到是哪个模块的问题。因此我们采用了分模块测试的思想, 即测试的时候每次只测试一个模块, 这样就可以更方便的定位到问题所在。

综上所述, 本次实验不仅锻炼了我画原理图、PCB 版图以及焊接的本领, 也锻炼了我设计电路模块以及代码实现的能力。感谢电子电路设计实验这门课程, 让我们在体验到电路设计乐趣的同时, 赋予了我们提升自我的机会。最后, 感谢老师们在实验过程中给予我们的细心指导与耐心讲解。在以后的学习生活中, 我会继续培养自己的核心素养, 在将理论与实践结合的过程中掌握更多专业要领, 为成为新型的高素质人才而不断前进。

十、参考资料

1. Analog Devices, *DS1302 Trickle-Charge Timekeeping Chip Datasheet*. <https://www.analog.com/media/en/technical-documentation/data-sheets/ds1302.pdf>
2. Arduino Libraries, *LiquidCrystal Library for Arduino*. <https://github.com/arduino-libraries/LiquidCrystal>
3. Texas Instruments, *LM35 Precision Centigrade Temperature Sensors Datasheet*. <https://www.ti.com/lit/ds/symlink/lm35.pdf>
4. PlatformIO Documentation, *Arduino Uno Board Configuration*. <https://docs.platformio.org/page/boards/atmelavr/uno.html>

一、附录：项目主要代码

1.1 main.cpp

```
1  #include <Arduino.h>
2  #include "ClockPlus.h"
3
4  void setup() {
5      clockPlusSetup();
6  }
7
8  void loop() {
9      clockPlusLoop();
10 }
```

1.2 ClockPlus.h

```
1  #ifndef CLOCK_PLUS_H
2  #define CLOCK_PLUS_H
3
4  #include <Arduino.h>
5  #include <DS1302.h>
6  #include <LiquidCrystal.h>
7  #include <EEPROM.h>
8
9  void clockPlusSetup();
10 void clockPlusLoop();
11
12 #endif
```

1.3 ClockPlus.cpp

```
1  #include "ClockPlus.h"
2
3  // =====
4  // ArduinoClockFancy
5  // Arduino UNO + DS1302 + LCD1602 + LM35 + 3 buttons + buzzer
6  // 保留原实验功能，并加入：
7  // 1. 多模式菜单
8  // 2. EEPROM 保存用户设置
9  // 3. 三组闹钟 + 工作日/周末模式 + 贪睡
10 // 4. 秒表、倒计时、番茄钟
11 // 5. LCD 自定义字符动画、温度趋势、温度历史条形图
12 // 6. 多种蜂鸣旋律、隐藏彩蛋
13 // =====
```

```
14
15 // ----- Pin map -----
16 LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // RS,E,D4,D5,D6,D7
17
18 const int PIN_CHOOSE = A0;
19 const int PIN_ADD     = A1;
20 const int PIN_MINUS   = A2;
21 const int PIN_LM35     = A3;
22 const int PIN_BUZZER = 13;
23
24 const uint8_t CE_PIN   = 8;
25 const uint8_t IO_PIN   = 9;
26 const uint8_t SCLK_PIN = 10;
27 DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN);
28
29 // ----- Data structures -----
30 enum Mode : uint8_t {
31     MODE_CLOCK = 0,
32     MODE_TEMP,
33     MODE_STOPWATCH,
34     MODE_TIMER,
35     MODE_POMODORO,
36     MODE_HISTORY,
37     MODE_COUNT
38 };
39
40 enum AlarmRepeat : uint8_t {
41     REPEAT Everyday = 0,
42     REPEAT_WORKDAY,
43     REPEAT_WEEKEND
44 };
45
46 struct AlarmSetting {
47     uint8_t hour;
48     uint8_t minute;
49     uint8_t enable;
50     uint8_t repeat;
51 };
52
53 struct Settings {
54     uint16_t magic;
55     AlarmSetting alarms[3];
56     int16_t tempAlarm10; // 温度阈值 * 10
57     uint8_t timerH;
58     uint8_t timerM;
59     uint8_t timerS;
60     uint8_t pomoworkMin;
61     uint8_t pomobreakMin;
62 };
```



```
63
64 const uint16_t EEPROM_MAGIC = 0x52A6;
65 Settings cfg;
66
67 // ----- State variables -----
68 Mode currentMode = MODE_CLOCK;
69 Mode lastMode = MODE_COUNT;
70
71 unsigned long lastDisplayMillis = 0;
72 unsigned long lastTempSampleMillis = 0;
73 unsigned long lastTempAlarmMillis = 0;
74 unsigned long lastAnimMillis = 0;
75 uint8_t animFrame = 0;
76
77 float currentTemp = 0.0;
78 float minTemp = 100.0;
79 float maxTemp = -100.0;
80 float previousTrendTemp = 0.0;
81 int8_t tempTrend = 0; // -1 down, 0 flat, 1 up
82
83 const uint8_t HISTORY_SIZE = 16;
84 float tempHistory[HISTORY_SIZE];
85 uint8_t tempHistoryCount = 0;
86 uint8_t tempHistoryIndex = 0;
87
88 long lastHourChimeKey = -1;
89 long lastAlarmKey[3] = {-1, -1, -1};
90
91 bool ringing = false;
92 uint8_t ringingType = 0; // 1 alarm, 2 timer, 3 pomo, 4 temp
93 unsigned long ringingStartMillis = 0;
94 uint8_t activeAlarmIndex = 0;
95 bool snoozeActive = false;
96 uint8_t snoozeHour = 0;
97 uint8_t snoozeMinute = 0;
98 long lastSnoozeKey = -1;
99
100 // Stopwatch
101 bool stopwatchRunning = false;
102 unsigned long stopwatchStartMillis = 0;
103 unsigned long stopwatchAccumMillis = 0;
104
105 // Countdown
106 bool timerRunning = false;
107 unsigned long timerRemainMillis = 0;
108 unsigned long timerLastMillis = 0;
109
110 // Pomodoro
111 bool pomoRunning = false;
```

```

112 bool pomoworkPhase = true;
113 uint8_t pomoround = 1;
114 unsigned long pomoremainmillis = 0;
115 unsigned long pomolastmillis = 0;
116
117 // LCD custom character mode: 0 none, 1 icons, 2 bars
118 uint8_t charSetMode = 0;
119
120 // ----- Musical notes -----
121 #define NOTE_C5 523
122 #define NOTE_D5 587
123 #define NOTE_E5 659
124 #define NOTE_F5 698
125 #define NOTE_G5 784
126 #define NOTE_A5 880
127 #define NOTE_B5 988
128 #define NOTE_C6 1047
129 #define NOTE_D6 1175
130 #define NOTE_E6 1319
131 #define NOTE_G6 1568
132
133 // ----- Utility -----
134 bool keyDown(int pin) {
135     return digitalRead(pin) == LOW;
136 }
137
138 bool keyPressed(int pin) {
139     if (digitalRead(pin) == LOW) {
140         delay(20);
141         if (digitalRead(pin) == LOW) {
142             while (digitalRead(pin) == LOW) delay(5);
143             return true;
144         }
145     }
146     return false;
147 }
148
149 void clearLine(uint8_t row) {
150     lcd.setCursor(0, row);
151     lcd.print(" ");
152 }
153
154 void print2(int n) {
155     if (n < 10) lcd.print('0');
156     lcd.print(n);
157 }
158
159 void print4(int n) {
160     if (n < 1000) lcd.print('0');

```

```

161     if (n < 100) lcd.print('0');
162     if (n < 10)  lcd.print('0');
163     lcd.print(n);
164 }
165
166 bool isLeapYear(int y) {
167     return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
168 }
169
170 int daysInMonth(int y, int m) {
171     switch (m) {
172         case 1: case 3: case 5: case 7: case 8: case 10: case 12: return 31;
173         case 4: case 6: case 9: case 11: return 30;
174         case 2: return isLeapYear(y) ? 29 : 28;
175     }
176     return 30;
177 }
178
179 void fixDate(int &y, int &m, int &d) {
180     if (y < 2000) y = 2000;
181     if (y > 2099) y = 2099;
182     if (m < 1) m = 12;
183     if (m > 12) m = 1;
184     int md = daysInMonth(y, m);
185     if (d < 1) d = md;
186     if (d > md) d = 1;
187 }
188
189 // Return 0..6 = Mon..Sun
190 int weekIndex(int y, int m, int d) {
191     if (m == 1) { m = 13; y--; }
192     else if (m == 2) { m = 14; y--; }
193     return (d + 2 * m + 3 * (m + 1) / 5 + y + y / 4 - y / 100 + y / 400) % 7;
194 }
195
196 const char* weekName(int y, int m, int d) {
197     static const char* names[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
198     return names[weekIndex(y, m, d)];
199 }
200
201 Time::Day ds1302DayFromDate(int y, int m, int d) {
202     switch (weekIndex(y, m, d)) {
203         case 0: return Time::kMonday;
204         case 1: return Time::kTuesday;
205         case 2: return Time::kWednesday;
206         case 3: return Time::kThursday;
207         case 4: return Time::kFriday;
208         case 5: return Time::kSaturday;
209         case 6: return Time::kSunday;

```

```

210     }
211     return Time::kMonday;
212 }
213
214 float readTemperature() {
215     int value = analogRead(PIN_LM35);
216     // LM35: 10 mV / degree C. Arduino UNO ADC: 0~1023 corresponds to 0~5V.
217     return 500.0 * value / 1023.0;
218 }
219
220 void sampleTemperature(bool force = false) {
221     if (!force && millis() - lastTempSampleMillis < 2000) return;
222     lastTempSampleMillis = millis();
223
224     float t = readTemperature();
225     currentTemp = t;
226     if (t > maxTemp) maxTemp = t;
227     if (t < minTemp) minTemp = t;
228
229     if (t > previousTrendTemp + 0.3) tempTrend = 1;
230     else if (t < previousTrendTemp - 0.3) tempTrend = -1;
231     else tempTrend = 0;
232     previousTrendTemp = t;
233
234     // 每次采样都更新历史, 16格循环
235     tempHistory[tempHistoryIndex] = t;
236     tempHistoryIndex = (tempHistoryIndex + 1) % HISTORY_SIZE;
237     if (tempHistoryCount < HISTORY_SIZE) tempHistoryCount++;
238 }
239
240 // ----- Custom characters -----
241 void loadIconChars() {
242     if (charSetMode == 1) return;
243     charSetMode = 1;
244
245     byte bell[8]  = {0x04,0x0E,0x0E,0x0E,0x1F,0x00,0x04,0x00};
246     byte up[8]    = {0x04,0x0E,0x15,0x04,0x04,0x04,0x04,0x00};
247     byte down[8]  = {0x04,0x04,0x04,0x04,0x15,0x0E,0x04,0x00};
248     byte flat[8]  = {0x00,0x00,0x00,0x1F,0x1F,0x00,0x00,0x00};
249     byte heart1[8]= {0x00,0x0A,0x1F,0x1F,0x0E,0x04,0x00,0x00};
250     byte heart2[8]= {0x00,0x00,0x0A,0x15,0x11,0x0A,0x04,0x00};
251     byte hourg[8] = {0x1F,0x11,0x0A,0x04,0x0A,0x11,0x1F,0x00};
252     byte note[8]  = {0x02,0x03,0x02,0x0E,0x1E,0x0C,0x00,0x00};
253
254     lcd.createChar(0, bell);
255     lcd.createChar(1, up);
256     lcd.createChar(2, down);
257     lcd.createChar(3, flat);
258     lcd.createChar(4, heart1);

```

```

259     lcd.createChar(5, heart2);
260     lcd.createChar(6, hourg);
261     lcd.createChar(7, note);
262 }
263
264 void loadBarChars() {
265     if (charSetMode == 2) return;
266     charSetMode = 2;
267
268     for (uint8_t level = 0; level < 8; level++) {
269         byte b[8];
270         for (uint8_t row = 0; row < 8; row++) {
271             b[row] = (row >= 7 - level) ? 0x1F : 0x00;
272         }
273         lcd.createChar(level, b);
274     }
275 }
276
277 // ----- Sound -----
278 void playToneBlocking(int freq, int ms) {
279     tone(PIN_BUZZER, freq, ms);
280     delay(ms + 20);
281     noTone(PIN_BUZZER);
282 }
283
284 void playClick() {
285     tone(PIN_BUZZER, NOTE_C6, 40);
286     delay(45);
287     noTone(PIN_BUZZER);
288 }
289
290 void playHourChime() {
291     playToneBlocking(NOTE_C6, 80);
292     playToneBlocking(NOTE_G6, 120);
293 }
294
295 void playAlarmMelody(uint8_t alarmIndex) {
296     if (alarmIndex == 0) {
297         int notes[] = {NOTE_C5, NOTE_E5, NOTE_G5, NOTE_C6};
298         for (uint8_t i = 0; i < 4; i++) playToneBlocking(notes[i], 90);
299     } else if (alarmIndex == 1) {
300         int notes[] = {NOTE_G5, NOTE_G5, NOTE_A5, NOTE_G5, NOTE_C6, NOTE_B5};
301         for (uint8_t i = 0; i < 6; i++) playToneBlocking(notes[i], 120);
302     } else {
303         int notes[] = {NOTE_C6, NOTE_B5, NOTE_A5, NOTE_G5, NOTE_A5, NOTE_B5, NOTE_C6};
304         for (uint8_t i = 0; i < 7; i++) playToneBlocking(notes[i], 70);
305     }
306 }
307

```



```

308 void playTimerDone() {
309     for (uint8_t i = 0; i < 3; i++) {
310         playToneBlocking(NOTE_E6, 100);
311         delay(40);
312     }
313 }
314
315 void playSecretTune() {
316     int notes[] = {NOTE_C5, NOTE_D5, NOTE_E5, NOTE_G5, NOTE_E5, NOTE_G5, NOTE_C6};
317     for (uint8_t i = 0; i < 7; i++) playToneBlocking(notes[i], 90);
318 }
319
320 void startRinging(uint8_t type, uint8_t alarmIdx = 0) {
321     ringing = true;
322     ringingType = type;
323     ringingStartMillis = millis();
324     activeAlarmIndex = alarmIdx;
325
326     if (type == 1) playAlarmMelody(alarmIdx);
327     else if (type == 2 || type == 3) playTimerDone();
328     else if (type == 4) playToneBlocking(NOTE_D6, 120);
329 }
330
331 void stopRinging() {
332     ringing = false;
333     ringingType = 0;
334     noTone(PIN_BUZZER);
335 }
336
337 // ----- EEPROM -----
338 void setDefaultSettings() {
339     cfg.magic = EEPROM_MAGIC;
340     cfg.alarms[0] = {7, 30, 1, REPEAT Everyday};
341     cfg.alarms[1] = {12, 0, 0, REPEAT WORKDAY};
342     cfg.alarms[2] = {22, 30, 0, REPEAT Everyday};
343     cfg.tempAlarm10 = 300;
344     cfg.timerH = 0;
345     cfg.timerM = 5;
346     cfg.timerS = 0;
347     cfg.pomoWorkMin = 25;
348     cfg.pomoBreakMin = 5;
349 }
350
351 void saveSettings() {
352     EEPROM.put(0, cfg);
353 }
354
355 void loadSettings() {
356     EEPROM.get(0, cfg);

```

```
357     if (cfg.magic != EEPROM_MAGIC || cfg.tempAlarm10 < 0 || cfg.tempAlarm10 > 990) {
358         setDefaultSettings();
359         saveSettings();
360     }
361     for (uint8_t i = 0; i < 3; i++) {
362         if (cfg.alarms[i].hour > 23) cfg.alarms[i].hour = 7;
363         if (cfg.alarms[i].minute > 59) cfg.alarms[i].minute = 0;
364         if (cfg.alarms[i].repeat > 2) cfg.alarms[i].repeat = REPEAT Everyday;
365     }
366     if (cfg.timerH > 23) cfg.timerH = 0;
367     if (cfg.timerM > 59) cfg.timerM = 5;
368     if (cfg.timerS > 59) cfg.timerS = 0;
369     if (cfg.pomoWorkMin < 1 || cfg.pomoWorkMin > 60) cfg.pomoWorkMin = 25;
370     if (cfg.pomoBreakMin < 1 || cfg.pomoBreakMin > 30) cfg.pomoBreakMin = 5;
371 }
372
373 // ----- Display screens -----
374 void displayClock(Time t) {
375     loadIconChars();
376     clearLine(0);
377     lcd.setCursor(0, 0);
378     print4(t.yr); lcd.print('-'); print2(t.mon); lcd.print('-'); print2(t.date);
379     lcd.print(' '); lcd.print(weekName(t.yr, t.mon, t.date));
380
381     clearLine(1);
382     lcd.setCursor(0, 1);
383     print2(t.hr); lcd.print(':'); print2(t.min); lcd.print(':'); print2(t.sec);
384
385     lcd.setCursor(9, 1);
386     if (cfg.alarms[0].enable || cfg.alarms[1].enable || cfg.alarms[2].enable) lcd.write((uint8_t)0);
387     else lcd.print(' ');
388
389     lcd.print(currentTemp, 1);
390     if (tempTrend > 0) lcd.write((uint8_t)1);
391     else if (tempTrend < 0) lcd.write((uint8_t)2);
392     else lcd.write((uint8_t)3);
393 }
394
395 void displayTempStats() {
396     loadIconChars();
397     clearLine(0);
398     lcd.setCursor(0, 0);
399     lcd.print("Temp "); lcd.print(currentTemp, 1);
400     if (tempTrend > 0) lcd.write((uint8_t)1);
401     else if (tempTrend < 0) lcd.write((uint8_t)2);
402     else lcd.write((uint8_t)3);
403
404     clearLine(1);
405     lcd.setCursor(0, 1);
```

```

406     lcd.print("H"); lcd.print(maxTemp, 1);
407     lcd.print(" L"); lcd.print(minTemp, 1);
408 }
409
410 void displayStopwatch() {
411     loadIconChars();
412     unsigned long elapsed = stopwatchAccumMillis;
413     if (stopwatchRunning) elapsed += millis() - stopwatchStartMillis;
414
415     unsigned long tenths = elapsed / 100;
416     uint8_t tenth = tenths % 10;
417     unsigned long totalSec = elapsed / 1000;
418     uint8_t sec = totalSec % 60;
419     uint8_t min = (totalSec / 60) % 60;
420     uint8_t hr = (totalSec / 3600) % 100;
421
422     clearLine(0);
423     lcd.setCursor(0, 0);
424     lcd.print("Stopwatch ");
425     lcd.print(stopwatchRunning ? "Run" : "Pause");
426
427     clearLine(1);
428     lcd.setCursor(0, 1);
429     print2(hr); lcd.print(':'); print2(min); lcd.print(':'); print2(sec); lcd.print('.'); lcd.print(tenth);
430 }
431
432 unsigned long presetTimerMillis() {
433     unsigned long s = (unsigned long)cfg.timerH * 3600UL + (unsigned long)cfg.timerM * 60UL + cfg.timerS;
434     if (s == 0) s = 60;
435     return s * 1000UL;
436 }
437
438 void displayTimer() {
439     loadIconChars();
440     unsigned long remain = timerRemainMillis;
441     unsigned long totalSec = remain / 1000;
442     uint8_t sec = totalSec % 60;
443     uint8_t min = (totalSec / 60) % 60;
444     uint8_t hr = (totalSec / 3600) % 100;
445
446     clearLine(0);
447     lcd.setCursor(0, 0);
448     lcd.write((uint8_t)6);
449     lcd.print(" Countdown ");
450     lcd.print(timerRunning ? "Run" : "Stop");
451
452     clearLine(1);
453     lcd.setCursor(0, 1);
454     print2(hr); lcd.print(':'); print2(min); lcd.print(':'); print2(sec);

```

```
455 }
456
457 void displayPomodoro() {
458     loadIconChars();
459     unsigned long totalSec = pomodoroRemainMillis / 1000;
460     uint8_t sec = totalSec % 60;
461     uint8_t min = (totalSec / 60) % 60;
462
463     clearLine(0);
464     lcd.setCursor(0, 0);
465     lcd.print("Pomo ");
466     lcd.print(pomodoroWorkPhase ? "Work " : "Break");
467     lcd.print(pomodoroRound); lcd.print("/4");
468     lcd.setCursor(15, 0);
469     lcd.write((uint8_t)(animFrame % 2 ? 4 : 5));
470
471     clearLine(1);
472     lcd.setCursor(0, 1);
473     print2(min); lcd.print(':'); print2(sec);
474     lcd.print(pomodoroRunning ? " Focus!" : " Pause ");
475 }
476
477 void displayHistory() {
478     loadBarChars();
479     clearLine(0);
480     lcd.setCursor(0, 0);
481     lcd.print("Temp History");
482
483     clearLine(1);
484     lcd.setCursor(0, 1);
485     if (tempHistoryCount == 0) {
486         lcd.print("No data");
487         return;
488     }
489
490     float lo = 100.0, hi = -100.0;
491     for (uint8_t i = 0; i < tempHistoryCount; i++) {
492         float v = tempHistory[i];
493         if (v < lo) lo = v;
494         if (v > hi) hi = v;
495     }
496     if (hi - lo < 0.5) hi = lo + 0.5;
497
498     for (uint8_t col = 0; col < 16; col++) {
499         if (col >= tempHistoryCount) {
500             lcd.print(' ');
501         } else {
502             uint8_t idx = (tempHistoryIndex + HISTORY_SIZE - tempHistoryCount + col) % HISTORY_SIZE;
503             float v = tempHistory[idx];
```

```

504     int level = (int)((v - lo) * 7.0 / (hi - lo) + 0.5);
505     if (level < 0) level = 0;
506     if (level > 7) level = 7;
507     lcd.write((uint8_t)level);
508 }
509 }
510 }
511
512 void displayMode() {
513     Time t = rtc.time();
514     switch (currentMode) {
515         case MODE_CLOCK: displayClock(t); break;
516         case MODE_TEMP: displayTempStats(); break;
517         case MODE_STOPWATCH: displayStopwatch(); break;
518         case MODE_TIMER: displayTimer(); break;
519         case MODE_POMODORO: displayPomodoro(); break;
520         case MODE_HISTORY: displayHistory(); break;
521         default: break;
522     }
523 }
524
525 // ----- Settings screens -----
526 const char* repeatName(uint8_t r) {
527     if (r == REPEAT_WORKDAY) return "Work";
528     if (r == REPEAT_WEEKEND) return "Wkend";
529     return "Every";
530 }
531
532 void displaySetting(uint8_t field, uint8_t alarmIndex) {
533     loadIconChars();
534     clearLine(0);
535     clearLine(1);
536     lcd.setCursor(0, 0);
537
538     if (field <= 4) {
539         lcd.print("Alarm"); lcd.print(alarmIndex + 1);
540         lcd.print(cfg.alarms[alarmIndex].enable ? " ON " : " OFF");
541         lcd.print(repeatName(cfg.alarms[alarmIndex].repeat));
542         lcd.setCursor(0, 1);
543         print2(cfg.alarms[alarmIndex].hour); lcd.print(':'); print2(cfg.alarms[alarmIndex].minute);
544         lcd.print(" Sel/+/- ");
545         if (field == 0) lcd.setCursor(5, 0); // alarm index
546         else if (field == 1) lcd.setCursor(7, 0); // on off
547         else if (field == 2) lcd.setCursor(0, 1); // hour
548         else if (field == 3) lcd.setCursor(3, 1); // minute
549         else lcd.setCursor(10, 0); // repeat
550     } else if (field == 5) {
551         lcd.print("Temp Alarm");
552         lcd.setCursor(0, 1);

```



```

553     lcd.print(cfg.tempAlarm10 / 10.0, 1); lcd.print("C");
554     lcd.setCursor(0, 1);
555 } else if (field >= 6 && field <= 8) {
556     lcd.print("Timer Preset");
557     lcd.setCursor(0, 1);
558     print2(cfg.timerH); lcd.print(':'); print2(cfg.timerM); lcd.print(':'); print2(cfg.timerS);
559     if (field == 6) lcd.setCursor(0, 1);
560     else if (field == 7) lcd.setCursor(3, 1);
561     else lcd.setCursor(6, 1);
562 } else if (field == 9) {
563     lcd.print("Pomo Work Min");
564     lcd.setCursor(0, 1);
565     print2(cfg.pomoWorkMin); lcd.print(" min");
566     lcd.setCursor(0, 1);
567 } else if (field == 10) {
568     lcd.print("Pomo Break Min");
569     lcd.setCursor(0, 1);
570     print2(cfg.pomoBreakMin); lcd.print(" min");
571     lcd.setCursor(0, 1);
572 }
573 lcd.cursor();
574 }
575
576 void enterSettings() {
577     uint8_t field = 0;
578     uint8_t alarmIndex = 0;
579     lcd.cursor();
580     playClick();
581
582     while (true) {
583         displaySetting(field, alarmIndex);
584
585         if (keyPressed(PIN_CHOOSE)) {
586             field++;
587             if (field > 10) break;
588         }
589         if (keyPressed(PIN_ADD)) {
590             AlarmSetting &a = cfg.alarms[alarmIndex];
591             switch (field) {
592                 case 0: alarmIndex = (alarmIndex + 1) % 3; break;
593                 case 1: a.enable = !a.enable; break;
594                 case 2: a.hour = (a.hour + 1) % 24; break;
595                 case 3: a.minute = (a.minute + 1) % 60; break;
596                 case 4: a.repeat = (a.repeat + 1) % 3; break;
597                 case 5: cfg.tempAlarm10 += 10; if (cfg.tempAlarm10 > 990) cfg.tempAlarm10 = 990; break;
598                 case 6: cfg.timerH = (cfg.timerH + 1) % 24; break;
599                 case 7: cfg.timerM = (cfg.timerM + 1) % 60; break;
600                 case 8: cfg.timerS = (cfg.timerS + 1) % 60; break;
601                 case 9: cfg.pomoWorkMin++; if (cfg.pomoWorkMin > 60) cfg.pomoWorkMin = 1; break;

```

```

602     case 10: cfg.pomoBreakMin++; if (cfg.pomoBreakMin > 30) cfg.pomoBreakMin = 1; break;
603 }
604 }
605 if (keyPressed(PIN_MINUS)) {
606     AlarmSetting &a = cfg.alarms[alarmIndex];
607     switch (field) {
608         case 0: alarmIndex = (alarmIndex + 2) % 3; break;
609         case 1: a.enable = !a.enable; break;
610         case 2: a.hour = (a.hour + 23) % 24; break;
611         case 3: a.minute = (a.minute + 59) % 60; break;
612         case 4: a.repeat = (a.repeat + 2) % 3; break;
613         case 5: cfg.tempAlarm10 -= 10; if (cfg.tempAlarm10 < 0) cfg.tempAlarm10 = 0; break;
614         case 6: cfg.timerH = (cfg.timerH + 23) % 24; break;
615         case 7: cfg.timerM = (cfg.timerM + 59) % 60; break;
616         case 8: cfg.timerS = (cfg.timerS + 59) % 60; break;
617         case 9: cfg.pomoWorkMin = (cfg.pomoWorkMin <= 1) ? 60 : cfg.pomoWorkMin - 1; break;
618         case 10: cfg.pomoBreakMin = (cfg.pomoBreakMin <= 1) ? 30 : cfg.pomoBreakMin - 1; break;
619     }
620 }
621 delay(80);
622 }
623
624 saveSettings();
625 timerRemainMillis = presetTimerMillis();
626 pomoRemainMillis = (unsigned long)cfg.pomoWorkMin * 60000UL;
627 lcd.noCursor();
628 lcd.clear();
629 playTimerDone();
630 }
631
632 void displayTimeSetting(uint8_t field, int y, int mon, int d, int h, int m, int s) {
633     clearLine(0); clearLine(1);
634     lcd.setCursor(0, 0);
635     print4(y); lcd.print('-'); print2(mon); lcd.print('-'); print2(d);
636     lcd.setCursor(0, 1);
637     print2(h); lcd.print(':'); print2(m); lcd.print(':'); print2(s); lcd.print(" Set");
638
639     switch (field) {
640         case 0: lcd.setCursor(0, 1); break;
641         case 1: lcd.setCursor(3, 1); break;
642         case 2: lcd.setCursor(6, 1); break;
643         case 3: lcd.setCursor(8, 0); break;
644         case 4: lcd.setCursor(5, 0); break;
645         case 5: lcd.setCursor(0, 0); break;
646     }
647     lcd.cursor();
648 }
649
650 void enterTimeSetting() {

```

```
651 loadIconChars();
652 Time t = rtc.time();
653 int y = t.yr, mon = t.mon, d = t.date, h = t.hr, m = t.min, s = t.sec;
654 fixDate(y, mon, d);
655 uint8_t field = 0;
656 lcd.cursor();
657 playClick();
658
659 while (true) {
660     fixDate(y, mon, d);
661     displayTimeSetting(field, y, mon, d, h, m, s);
662     if (keyPressed(PIN_CHOOSE)) {
663         field++;
664         if (field > 5) break;
665     }
666     if (keyPressed(PIN_ADD)) {
667         switch (field) {
668             case 0: h = (h + 1) % 24; break;
669             case 1: m = (m + 1) % 60; break;
670             case 2: s = (s + 1) % 60; break;
671             case 3: d++; break;
672             case 4: mon++; break;
673             case 5: y++; break;
674         }
675     }
676     if (keyPressed(PIN_MINUS)) {
677         switch (field) {
678             case 0: h = (h + 23) % 24; break;
679             case 1: m = (m + 59) % 60; break;
680             case 2: s = (s + 59) % 60; break;
681             case 3: d--; break;
682             case 4: mon--; break;
683             case 5: y--; break;
684         }
685     }
686     delay(80);
687 }
688
689 fixDate(y, mon, d);
690 rtc.time(Time(y, mon, d, h, m, s, ds1302DayFromDate(y, mon, d)));
691 lcd.noCursor();
692 lcd.clear();
693 playTimerDone();
694 }
695
696 // ----- Alarm logic -----
697 bool alarmRepeatMatches(uint8_t repeat, Time t) {
698     int w = weekIndex(t.yr, t.mon, t.date); // Mon=0, Sun=6
699     if (repeat == REPEAT Everyday) return true;
```

```
700     if (repeat == REPEAT_WORKDAY) return w >= 0 && w <= 4;
701     if (repeat == REPEAT_WEEKEND) return w == 5 || w == 6;
702     return true;
703 }
704
705 long dayMinuteKey(Time t, uint8_t h, uint8_t m) {
706     return (long)t.yr * 100000000L + (long)t.mon * 1000000L + (long)t.date * 10000L + (long)h * 100L + m;
707 }
708
709 void checkHourChime(Time t) {
710     long key = (long)t.yr * 1000000L + (long)t.mon * 10000L + (long)t.date * 100L + t.hr;
711     if (t.min == 0 && t.sec == 0 && key != lastHourChimeKey) {
712         lastHourChimeKey = key;
713         playHourChime();
714     }
715 }
716
717 void checkAlarms(Time t) {
718     for (uint8_t i = 0; i < 3; i++) {
719         AlarmSetting &a = cfg.alarms[i];
720         if (!a.enable) continue;
721         long key = dayMinuteKey(t, a.hour, a.minute);
722         if (t.hr == a.hour && t.min == a.minute && t.sec == 0 && alarmRepeatMatches(a.repeat, t) && key !=
            lastAlarmKey[i]) {
723             lastAlarmKey[i] = key;
724             startRinging(1, i);
725         }
726     }
727
728     if (snoozeActive) {
729         long key = dayMinuteKey(t, snoozeHour, snoozeMinute);
730         if (t.hr == snoozeHour && t.min == snoozeMinute && t.sec == 0 && key != lastSnoozeKey) {
731             lastSnoozeKey = key;
732             snoozeActive = false;
733             startRinging(1, activeAlarmIndex);
734         }
735     }
736 }
737
738 void checkTemperatureAlarm() {
739     float alarmTemp = cfg.tempAlarm10 / 10.0;
740     if (currentTemp < alarmTemp) return;
741     if (millis() - lastTempAlarmMillis < 5000) return;
742     lastTempAlarmMillis = millis();
743
744     float over = currentTemp - alarmTemp;
745     if (over >= 10.0) {
746         playToneBlocking(NOTE_E6, 80);
747         playToneBlocking(NOTE_E6, 80);
```

```

748     playToneBlocking(NOTE_E6, 80);
749 } else if (over >= 5.0) {
750     playToneBlocking(NOTE_D6, 120);
751     playToneBlocking(NOTE_D6, 120);
752 } else {
753     startRinging(4);
754 }
755 }
756
757 void handleRingingButtons() {
758     if (!ringing) return;
759
760     loadIconChars();
761     clearLine(0); clearLine(1);
762     lcd.setCursor(0, 0);
763     if (ringingType == 1) {
764         lcd.write((uint8_t)0); lcd.print(" Alarm"); lcd.print(activeAlarmIndex + 1);
765     } else if (ringingType == 2) {
766         lcd.write((uint8_t)6); lcd.print(" Timer Done");
767     } else if (ringingType == 3) {
768         lcd.print("Pomodoro Done");
769     } else {
770         lcd.print("Temp Warning!");
771     }
772     lcd.setCursor(0, 1);
773     lcd.print("SEL off +5 -10");
774
775     if (keyPressed(PIN_CHOOSE)) {
776         stopRinging();
777         lcd.clear();
778         return;
779     }
780     if (keyPressed(PIN_ADD)) {
781         Time t = rtc.time();
782         int total = t.hr * 60 + t.min + 5;
783         snoozeHour = (total / 60) % 24;
784         snoozeMinute = total % 60;
785         snoozeActive = true;
786         stopRinging();
787         lcd.clear();
788         return;
789     }
790     if (keyPressed(PIN_MINUS)) {
791         Time t = rtc.time();
792         int total = t.hr * 60 + t.min + 10;
793         snoozeHour = (total / 60) % 24;
794         snoozeMinute = total % 60;
795         snoozeActive = true;
796         stopRinging();

```

```

797     lcd.clear();
798     return;
799 }
800
801 // 20秒后自动停止蜂鸣界面，避免一直阻塞展示
802 if (millis() - ringingStartMillis > 20000UL) {
803     stopRinging();
804     lcd.clear();
805 }
806 }
807
808 // ----- Mode logic -----
809 void nextMode() {
810     currentMode = (Mode)((currentMode + 1) % MODE_COUNT);
811     lcd.clear();
812     charSetMode = 0;
813 }
814
815 void prevMode() {
816     currentMode = (Mode)((currentMode + MODE_COUNT - 1) % MODE_COUNT);
817     lcd.clear();
818     charSetMode = 0;
819 }
820
821 void updateCountdown() {
822     if (!timerRunning) return;
823     unsigned long now = millis();
824     unsigned long delta = now - timerLastMillis;
825     timerLastMillis = now;
826     if (delta >= timerRemainMillis) {
827         timerRemainMillis = 0;
828         timerRunning = false;
829         startRinging(2);
830     } else {
831         timerRemainMillis -= delta;
832     }
833 }
834
835 void updatePomodoro() {
836     if (!pomoRunning) return;
837     unsigned long now = millis();
838     unsigned long delta = now - pomoLastMillis;
839     pomoLastMillis = now;
840
841     if (delta >= pomoRemainMillis) {
842         pomoRunning = false;
843         startRinging(3);
844         if (pomoWorkPhase) {
845             pomoWorkPhase = false;

```



```

846     pomoRemainMillis = (unsigned long)cfg.pomoBreakMin * 60000UL;
847 } else {
848     pomoWorkPhase = true;
849     pomoRound++;
850     if (pomoRound > 4) pomoRound = 1;
851     pomoRemainMillis = (unsigned long)cfg.pomoWorkMin * 60000UL;
852 }
853 } else {
854     pomoRemainMillis -= delta;
855 }
856 }
857
858 void showSecret() {
859     loadIconChars();
860     lcd.clear();
861     lcd.setCursor(0, 0);
862     lcd.print(" Secret Mode ");
863     lcd.write((uint8_t)4);
864     lcd.setCursor(0, 1);
865     lcd.print("Hello Teacher!");
866     playSecretTune();
867     delay(1000);
868     lcd.clear();
869 }
870
871 void handleButtons() {
872     // 组合键优先
873     if (keyDown(PIN_ADD) && keyDown(PIN_MINUS)) {
874         delay(50);
875         if (keyDown(PIN_ADD) && keyDown(PIN_MINUS)) {
876             while (keyDown(PIN_ADD) || keyDown(PIN_MINUS)) delay(5);
877             enterSettings();
878             return;
879         }
880     }
881
882     if (keyDown(PIN_CHOOSE) && keyDown(PIN_ADD)) {
883         delay(80);
884         if (keyDown(PIN_CHOOSE) && keyDown(PIN_ADD)) {
885             while (keyDown(PIN_CHOOSE) || keyDown(PIN_ADD)) delay(5);
886             showSecret();
887             return;
888         }
889     }
890
891     if (keyPressed(PIN_ADD)) {
892         nextMode();
893         playClick();
894         return;

```

```

895     }
896     if (keyPressed(PIN_MINUS)) {
897         if (currentMode == MODE_STOPWATCH) {
898             stopwatchRunning = false;
899             stopwatchAccumMillis = 0;
900         } else if (currentMode == MODE_TIMER) {
901             timerRunning = false;
902             timerRemainMillis = presetTimerMillis();
903         } else if (currentMode == MODE_POMODORO) {
904             pomoRunning = false;
905             pomoWorkPhase = true;
906             pomoRound = 1;
907             pomoRemainMillis = (unsigned long)cfg.pomoWorkMin * 60000UL;
908         } else {
909             prevMode();
910         }
911         playClick();
912         return;
913     }
914     if (keyPressed(PIN_CHOOSE)) {
915         if (currentMode == MODE_CLOCK) {
916             enterTimeSetting();
917         } else if (currentMode == MODE_STOPWATCH) {
918             if (stopwatchRunning) {
919                 stopwatchAccumMillis += millis() - stopwatchStartMillis;
920                 stopwatchRunning = false;
921             } else {
922                 stopwatchStartMillis = millis();
923                 stopwatchRunning = true;
924             }
925         } else if (currentMode == MODE_TIMER) {
926             if (timerRemainMillis == 0) timerRemainMillis = presetTimerMillis();
927             timerRunning = !timerRunning;
928             timerLastMillis = millis();
929         } else if (currentMode == MODE_POMODORO) {
930             pomoRunning = !pomoRunning;
931             pomoLastMillis = millis();
932         }
933         playClick();
934         return;
935     }
936 }
937
938 // ----- Setup and loop -----
939 void splash() {
940     loadIconChars();
941     lcd.clear();
942     lcd.setCursor(0, 0);
943     lcd.print("Clock Fancy ");

```

```

944     lcd.write((uint8_t)4);
945     lcd.setCursor(0, 1);
946     lcd.print("Loading");
947     for (uint8_t i = 0; i < 6; i++) {
948         lcd.print('.');
949         tone(PIN_BUZZER, NOTE_C5 + i * 80, 40);
950         delay(120);
951     }
952     noTone(PIN_BUZZER);
953     lcd.clear();
954 }
955
956 void clockPlusSetup() {
957     pinMode(PIN_CHOOSE, INPUT_PULLUP);
958     pinMode(PIN_ADD, INPUT_PULLUP);
959     pinMode(PIN_MINUS, INPUT_PULLUP);
960     pinMode(PIN_BUZZER, OUTPUT);
961     pinMode(PIN_LM35, INPUT);
962
963     lcd.begin(16, 2);
964     lcd.clear();
965
966     rtc.writeProtect(false);
967     rtc.halt(false);
968
969     // 第一次校时使用: 取消下一行注释, 改成当前时间, 烧录一次;
970     // rtc.time(Time(2026, 5, 26, 12, 0, 0, Time::kTuesday));
971
972     loadSettings();
973     timerRemainMillis = presetTimerMillis();
974     pomoRemainMillis = (unsigned long)cfg.pomoWorkMin * 60000UL;
975     sampleTemperature(true);
976     for (uint8_t i = 0; i < HISTORY_SIZE; i++) tempHistory[i] = currentTemp;
977     tempHistoryCount = HISTORY_SIZE;
978
979     splash();
980 }
981
982 void clockPlusLoop() {
983     if (ringing) {
984         handleRingingButtons();
985         return;
986     }
987
988     Time t = rtc.time();
989
990     sampleTemperature(false);
991     updateCountdown();
992     updatePomodoro();

```

```
993
994  if (millis() - lastAnimMillis >= 500) {
995      lastAnimMillis = millis();
996      animFrame++;
997  }
998
999  handleButtons();
1000
1001  checkHourChime(t);
1002  checkAlarms(t);
1003  checkTemperatureAlarm();
1004
1005  if (currentMode != lastMode) {
1006      lastMode = currentMode;
1007      lcd.clear();
1008      charSetMode = 0;
1009  }
1010
1011  if (millis() - lastDisplayMillis >= 250) {
1012      lastDisplayMillis = millis();
1013      displayMode();
1014  }
1015 }
```